

Intelligenza Artificiale

Guida Completa di Studio

Prolog · ASP/Clingo · CLIPS · Algoritmi di Ricerca

Alessandro Livero

29 maggio 2026

Indice

I	Prolog – Programmazione Logica	4
1	Fondamenti di Prolog	4
1.1	Cos'è Prolog	4
1.2	I Tre Elementi Base	4
1.3	Unificazione	4
1.4	Backtracking	4
1.5	Negazione per Fallimento ($\backslash +$)	5
1.6	Non-Monotonic Reasoning (NMR) – file <code>nmr.pl</code>	5
2	Liste in Prolog	5
2.1	Rappresentazione	5
2.2	Pattern: Ricorsione su Liste	6
2.3	Predicati su Liste – file <code>lista.pl</code>	6
2.3.1	<code>tuttiPositivi/1</code> – Controllo Uniforme	6
2.3.2	<code>ContaPositivi/2</code> – Conteggio	6
2.4	Inversione di Lista – file <code>invertilista.pl</code>	7
2.5	Operazioni su Insiemi – file <code>listeinsiemi.pl</code>	7
3	Il Cut (!)	7
3.1	Cos'è il Cut	7
3.2	Cut Verde vs Cut Rosso	8
3.3	Esempio: <code>contaPositivi</code> con Cut – file <code>listeconcut.pl</code>	8
3.4	Esempio: <code>sottolista</code> con Cut	8
4	Ricorsione e Alberi Genealogici – file <code>antenati.pl</code>	8
4.1	Predicati Genealogici	8
4.2	Antenato: Ricorsione su Relazioni	9
5	Predicati Higher-Order: <code>findall</code>, <code>call</code>	9
5.1	<code>findall/3</code>	9
5.2	<code>call/N</code> e Meta-predicati	10

6	A* e IDA* in Prolog – file <code>astar_idastar.pl</code>	10
6.1	Il Problema della Ricerca nello Spazio degli Stati	10
6.2	A* (A-Star)	10
6.2.1	Idea Chiave	10
6.2.2	Implementazione Prolog	10
6.2.3	Perché il Goal si Controlla all'Estrazione?	11
6.3	IDA* (Iterative Deepening A*)	11
6.3.1	Idea Chiave	11
6.4	Interfaccia Generica	12
6.5	Dominio 1: Labirinto (5×5)	12
6.6	Dominio 2: Puzzle dell'8	12
6.7	Confronto A* vs IDA*	13
II	Answer Set Programming (Clingo)	13
7	Fondamenti di ASP	13
7.1	Paradigma	13
7.2	Costrutti Fondamentali	14
8	Progetto Torneo Pokémon – <code>torneo.lp</code>	14
8.1	Sistema Svizzero	14
8.2	Normalizzazione: <code>pid</code> e <code>played_already</code>	14
8.3	Generazione	15
8.4	Vincoli Hard	15
8.5	Ottimizzazione	15
8.6	Istanze	16
III	CLIPS – Expert System	16
9	Fondamenti di CLIPS	16
9.1	Cos'è CLIPS	16
9.2	Ciclo di Inferenza	16
9.3	Costrutti Base	16
10	Progetto Wumpus – <code>agent.clp</code> + <code>wumpus.py</code>	17
10.1	Architettura Generale	17
10.2	Moduli dell'Agente CF	17
10.3	Template Principali	17
10.4	Certainty Factors (CF)	18
10.4.1	Esempio di regola CF – INFER	18
10.4.2	Classificazione delle Celle	19
10.5	Gerarchia delle Azioni – DECIDE	19
10.6	Agente Greedy vs Agente CF	20
IV	Schema Riassuntivo per l'Esame	20
11	Confronto tra i Paradigmi	20

12 Domande di Esame Tipiche	20
12.1 Su Prolog	20
12.2 Su ASP/Clingo	21
12.3 Su CLIPS	21
13 Formule da Ricordare	22

Parte I

Prolog – Programmazione Logica

1 Fondamenti di Prolog

1.1 Cos'è Prolog

Prolog (**P**rogramming in **L**ogic) è un linguaggio di programmazione **dichiarativo**: si descrive *cosa* è vero, non *come* calcolarlo. Il motore di Prolog (l'interprete) si occupa di trovare le soluzioni tramite **risoluzione SLD** (Selective Linear Definite clause resolution) e **backtracking**.

1.2 I Tre Elementi Base

Elemento	Sintassi	Significato	Esempio
Fatto	<code>pred(arg).</code>	Sempre vero	<code>gatto(tom).</code>
Regola	<code>h :- b1, b2.</code>	<code>h</code> è vero se <code>b1</code> e <code>b2</code>	<code>graffia(X) :- gatto(X).</code>
Query	<code>?- goal.</code>	Chiedi se è vero	<code>?- gatto(tom).</code>

Regola d'oro: lettere maiuscole

In Prolog tutto ciò che inizia con **lettera maiuscola** o **underscore** `_` è una **variabile**. Tutto il resto (lettere minuscole, numeri, simboli) è una **costante** (atomo o numero).

```
1 gatto(tom).           % tom e' una costante (atomo)
2 graffia(X) :- gatto(X). % X e' una variabile
```

1.3 Unificazione

L'unificazione è il meccanismo fondamentale di Prolog: due termini si **unificano** se esiste un'assegnazione di variabili che li rende identici.

Esempi di unificazione

```
?- gatto(X) = gatto(tom).      % X = tom (successo)
?- f(X, b) = f(a, Y).          % X = a, Y = b (successo)
?- f(X) = g(X).                % fallisce (f != g)
?- X = f(X).                   % X = f(f(f(...))) (occur check)
```

1.4 Backtracking

Quando una query fallisce o si vogliono più soluzioni, Prolog torna indietro (**backtrack**) e prova la prossima clausola disponibile.

```
1 uccello(titty).
2 pinguino(tux).
3 uccello(X) :- pinguino(X).    % un pinguino e' un uccello
```

```

4 vola(X) :- uccello(X), \+ pinguino(X).
5

```

Traccia di ?- vola(titty):

1. Cerca clausola con testa vola(?): trova vola(X) :- uccello(X), \+ pinguino(X). Unifica X = titty.
2. Prova uccello(titty): successo (è un fatto).
3. Prova \+ pinguino(titty): prova pinguino(titty) → fallisce → \+ succede. **Risultato: vero.**

Traccia di ?- vola(tux):

1. Unifica X = tux. Prova uccello(tux): non è un fatto diretto.
2. Prova regola: uccello(X) :- pinguino(X), X=tux. pinguino(tux) → successo. Quindi uccello(tux) succede.
3. Prova \+ pinguino(tux): pinguino(tux) → successo → \+ **fallisce. Risultato: falso.**

1.5 Negazione per Fallimento (\+)

Negazione per Fallimento (NAF)

\+ Goal è vero se Goal **fallisce** (non è dimostrabile). Implementa la **Closed World Assumption (CWA)**: ciò che non è derivabile si assume falso.

Non è la negazione logica classica: \+ p(X) non significa “esiste una prova che p(X) è falso”, ma “non riesco a provare p(X)”.

1.6 Non-Monotonic Reasoning (NMR) – file `nmr.pl`

```

1 % Il paradosso del pinguino
2 uccello(X) :- pinguino(X).
3 vola(X)    :- uccello(X), \+ pinguino(X).
4 uccello(titty).
5 pinguino(tux).
6 pinguino(titty).    % titty e' anche un pinguino!

```

Il ragionamento non-monotonico: normalmente gli uccelli volano. Ma aggiungere il fatto `pinguino(titty)` **ritira** la conclusione `vola(titty)`, anche se era stata derivata prima. Il sistema è *non-monotono*: aggiungere informazioni può eliminare conclusioni.

Questo è diverso dalla logica classica (monotona) dove aggiungere assiomi non toglie mai teoremi.

2 Liste in Prolog

2.1 Rappresentazione

Una lista è o la lista vuota `[]` oppure una coppia `[Testa|Coda]` dove **Testa** è il primo elemento e **Coda** è il resto (altra lista).

```

1 [1, 2, 3]      % lista di 3 elementi
2 [H|T]         % H = 1, T = [2, 3]
3 [A, B | R]    % A = 1, B = 2, R = [3]
4 []           % lista vuota

```

2.2 Pattern: Ricorsione su Liste

Il pattern universale per lavorare su liste in Prolog:

```

1 predicato([]).           % caso base: lista vuota
2 predicato([Head|Tail]) :- % passo ricorsivo
3     ... Head ...         % fai qualcosa con la testa
4     predicato(Tail).      % ricorsione sulla coda

```

2.3 Predicati su Liste – file lista.pl

2.3.1 tuttiPositivi/1 – Controllo Uniforme

```

1 tuttiPositivi([]).           % lista vuota: ok
2 tuttiPositivi([Head|Tail]) :-
3     atomic(Head), Head > 0,   % elemento > 0
4     tuttiPositivi(Tail).
5 tuttiPositivi([Head|Tail]) :-
6     is_list(Head),           % se e' una sottolista
7     tuttiPositivi(Head),      % controlla la sottolista
8     tuttiPositivi(Tail).

```

Gestisce liste annidate: `tuttiPositivi([1, [2, 3], 4])` → vero. Il predicato `atomic/1` controlla che non sia una lista; `is_list/1` controlla che sia una lista.

2.3.2 ContaPositivi/2 – Conteggio

```

1 ContaPositivi([], 0).
2 ContaPositivi([Head|Tail], Risultato) :-
3     atomic(Head), Head > 0,
4     ContaPositivi(Tail, Tot),
5     Risultato is Tot + 1.
6 ContaPositivi([Head|Tail], Risultato) :-
7     atomic(Head), Head <= 0,           % elemento <= 0: non conta
8     ContaPositivi(Tail, Risultato).
9 ContaPositivi([Head|Tail], Risultato) :-
10    is_list(Head),                     % sottolista: conta
11    ricorsivamente
12    ContaPositivi(Head, Tot1),
13    ContaPositivi(Tail, Tot2),
14    Risultato is Tot1 + Tot2.

```

Attenzione: is per l'aritmetica

In Prolog `X = 3 + 2` **non calcola**: unifica X con il termine `+(3,2)`. Per valutare, usare `X is 3 + 2` (`X = 5`).

2.4 Inversione di Lista – file invertilista.pl

```

1 % Versione naive: O(n^2)
2 % inverti_naive([1,2,3], X) => X = [3,2,1]
3
4 % Versione efficiente con accumulatore: O(n)
5 inverti(Lista, LInv) :- inv(Lista, [], LInv).
6
7 inv([], Acc, Acc).                % lista esaurita: l'accumulatore E' il
    risultato
8 inv([H|T], Acc, LInv) :-
9     inv(T, [H|Acc], LInv).        % metti H in testa all'accumulatore (
    inverti)

```

Traccia di `inv([1,2,3], [], Inv)`:

`inv([1,2,3], []) → inv([2,3], [1]) → inv([3], [2,1]) → inv([], [3,2,1])`
`→ Inv = [3,2,1]`

Il **pattern dell'accumulatore** è fondamentale: invece di ricostruire la lista durante il ritorno dalla ricorsione (lento), si costruisce il risultato *in avanti* passandolo come parametro aggiuntivo.

2.5 Operazioni su Insiemi – file listeinsiemi.pl

```

1 % Intersezione: elementi di L1 che stanno anche in L2
2 intersez([], _, []).
3 intersez(_, [], []).
4 intersez([H|T], L2, [H|R]) :- member(H, L2), !, intersez(T, L2, R).
5 intersez([_|T], L2, R)      :- intersez(T, L2, R).
6
7 % Unione: tutti gli elementi di L1 e L2 senza duplicati
8 unione([], L2, L2) :- !.
9 unione(L1, [], L1) :- !.
10 unione([H|T], L2, R)      :- member(H, L2), !, unione(T, L2, R).
11 unione([H|T], L2, [H|R])  :- unione(T, L2, R).
12
13 % Differenza: elementi di L1 non presenti in L2
14 diff([], _, []) :- !.
15 diff([H|T], L2, [H|R]) :- \+ member(H, L2), !, diff(T, L2, R).
16 diff([_|T], L2, R)      :- diff(T, L2, R).

```

3 Il Cut (!)

3.1 Cos'è il Cut

Il cut (!) è un predicato speciale che, quando viene incontrato, **congela** le scelte già fatte: elimina i punti di backtracking creati dalla clausola corrente e dalle sue sorelle.

Effetto del Cut

Quando ! viene eseguito nella clausola `h :- b1, !, b2`:

1. Tutte le clausole alternative per `h` vengono **scartate**.
2. I punti di backtracking creati da `b1` vengono **eliminati**.

3. Se **b2** fallisce, la query per **h** fallisce definitivamente (non prova le clausole successive).

3.2 Cut Verde vs Cut Rosso

Tipo	Effetto sulla semantica	Uso
Cut verde	Non cambia il significato logico	Solo efficienza
Cut rosso	Cambia il significato (rimuove soluzioni)	Esprimere priorità

3.3 Esempio: contaPositivi con Cut – file listeconcut.pl

```

1 contaPositivi([], 0).
2 contaPositivi([Head|Tail], Risultato) :-
3     Head > 0, !,                                % se Head > 0, CUT: evita di provare
4     le clausole sotto
5     contaPositivi(Tail, Tot),
6     Risultato is Tot + 1.
7 contaPositivi([_|Tail], Tot) :-                % questa clausola vale solo se Head
8     <= 0
9     contaPositivi(Tail, Tot).
```

Senza cut: Prolog proverebbe tutte e 3 le clausole per ogni elemento. **Con cut:** se **Head** > 0 riesce, si esegue solo la seconda clausola (quella con il cut), e la terza non viene mai provata. Se **Head** > 0 fallisce, si salta alla terza clausola. Questo è un **cut rosso**: modifica il comportamento (la terza clausola non si attiva mai quando **Head** > 0).

Altro uso del cut: prova/1

```

1 prova(X) :- member(X, [1,2,3,4,5]), !, member(X, [4,5,10,50]).
2 prova(152).
```

?- prova(X). trova il primo X in [1,2,3,4,5] (= 1), poi il cut impedisce di tornare sulla member. Allora prova member(1, [4,5,10,50]) → fallisce. Il cut impedisce anche di provare la seconda clausola prova(152). Risultato: **fallisce**. Senza il cut otterrebbe X = 4 o X = 5.

3.4 Esempio: sottolista con Cut

```

1 sottolista([Head|_], Head) :- is_list(Head), !. % primo elemento lista:
2     fermati
3 sottolista([_|Tail], R) :- sottolista(Tail, R). % altrimenti continua
```

Il cut garantisce che quando si trova il primo elemento lista, non si cerchino altre soluzioni nella stessa posizione.

4 Ricorsione e Alberi Genealogici – file antenati.pl

4.1 Predicati Genealogici

```

1 genitore(edoardo, gianni). % ... altri fatti ...
2 genitore(gianni, margherita).
3 genitore(margherita, johnElkann).
4
5 nonno(X, Y) :- genitore(X, Z), genitore(Z, Y).

```

Traccia di ?- nonno(edoardo, margherita):

1. Unifica X=edoardo, Y=margherita.
2. Cerca Z: genitore(edoardo, Z) → Z=gianni (primo risultato).
3. Controlla: genitore(gianni, margherita) → successo!

4.2 Antenato: Ricorsione su Relazioni

```

1 % Attenzione: questa definizione ha un bug!
2 antenato(X, Y) :- antenato(X, Z), genitore(Z, Y).

```

Bug: ricorsione a sinistra infinita

La definizione sopra va in loop infinito perché la prima chiamata a **antenato** genera subito un'altra chiamata a **antenato** senza caso base. La versione corretta dovrebbe essere:

```

1 antenato(X, Y) :- genitore(X, Y). % caso base
2 antenato(X, Y) :- genitore(X, Z), antenato(Z, Y). % ricorsione

```

La ricorsione **a destra** (il predicato ricorsivo appare dopo una chiamata a un fatto) termina correttamente.

5 Predicati Higher-Order: findall, call

5.1 findall/3

findall(+Template, +Goal, -Lista)

Raccoglie in Lista tutti i valori di Template per cui Goal è soddisfatto. Se Goal non ha soluzioni, Lista = [].

Non dà mai errore: se non trova niente, restituisce lista vuota.

```

1 haTipo(pichu, elettro).
2 haTipo(emboar, fuoco).
3 haTipo(volcanion, fuoco).
4 haTipo(volcanion, acqua).
5
6 % ?- findall(P, haTipo(P, fuoco), Lista).
7 % Lista = [emboar, volcanion]
8
9 % ?- findall(P-T, haTipo(P, T), Coppie).
10 % Coppie = [pichu-elettro, emboar-fuoco, volcanion-fuoco, volcanion-acqua]

```

5.2 call/N e Meta-predicati

`call(Goal)` esegue `Goal` come se fosse scritto direttamente. `call(Goal, Arg1)` aggiunge `Arg1` come argomento a `Goal`.

```

1 :- meta_predicate a_star(1, 3, 2, +, -, -, -).
2 % significa: il primo arg e' un predicato che call chiamera' con 1 extra
   arg
3
4 % Uso: a_star(maze_goal, maze_successors, maze_heuristic, Start, ...)
5 % internamente chiama: call(maze_goal, State)
6 %                       call(maze_successors, State, NS, Cost)
7 %                       call(maze_heuristic, State, H)

```

6 A* e IDA* in Prolog – file `astar_idastar.pl`

6.1 Il Problema della Ricerca nello Spazio degli Stati

Dato uno **stato iniziale**, un insieme di **operatori** (mosse), e una **condizione di goal**, trovare la sequenza di operatori che porta dallo stato iniziale a uno stato goal con **costo minimo**.

6.2 A* (A-Star)

6.2.1 Idea Chiave

A* esplora sempre il nodo con il valore $f = g + h$ minimo:

- $g(n)$: costo reale dallo stato iniziale a n
- $h(n)$: stima euristica (ammissibile: $h(n) \leq h^*(n)$) da n al goal
- $f(n) = g(n) + h(n)$: stima del costo totale attraverso n

6.2.2 Implementazione Prolog

```

1 % Struttura nodo nella frontiera: f(ValoreF, G, Stato, PercorsoInverso)
2 % PercorsoInverso e' la lista degli stati percorsi, in ordine inverso (
   efficiente)
3
4 astar(Domain, Start, Path, Cost, Expanded) :-
5     heuristic(Domain, Start, H),
6     % frontiera iniziale: un solo nodo con f=h, g=0
7     astar_loop(Domain, [f(H, 0, Start, [Start])], [], Path, Cost, 0,
        Expanded).
8
9 % Caso base: il nodo in testa e' il goal -> SOLUZIONE TROVATA
10 astar_loop(Domain, [f(_, G, State, RevPath)|_], _, Path, G, Exp, Exp) :-
11     goal(Domain, State),
12     reverse(RevPath, Path).    % inverte il percorso costruito al
        contrario
13
14 % Caso ricorsivo: espansione
15 astar_loop(Domain, [f(_, G, State, RevPath)|Open], Closed, Path, Cost,
        Exp0, Exp) :-
16     \+ goal(Domain, State),

```

```

17 (member(State, Closed) -> % gia' espanso: scarta (lazy
    deletion)
18     astar_loop(Domain, Open, Closed, Path, Cost, Exp0, Exp)
19 ;
20 Exp1 is Exp0 + 1,
21 findall(f(F1, G1, S1, [S1|RevPath]),
22     (move(Domain, State, S1, SC), % genera successori
23         \+ member(S1, Closed),
24         G1 is G + SC,
25         heuristic(Domain, S1, H1),
26         F1 is G1 + H1),
27     Successors),
28     insert_all(Successors, Open, Open1), % inserimento ordinato per F
29     astar_loop(Domain, Open1, [State|Closed], Path, Cost, Exp1, Exp)
30 ).

```

6.2.3 Perché il Goal si Controlla all'Estrazione?

Si controlla il goal quando il nodo è **estratto** (in testa alla lista), non quando viene inserito. Questo garantisce l'ottimalità: quando un nodo goal è in testa alla frontiera (ordinata per f), il suo g è il minimo possibile.

6.3 IDA* (Iterative Deepening A*)

6.3.1 Idea Chiave

Esegue DFS iterative con soglie t crescenti su f :

1. Prima iterazione: $t_0 = h(\text{start})$
2. DFS: espande nodi con $f \leq t$; se $f > t$ ritaglia (cutoff)
3. Se la DFS non trova il goal: $t_{\text{new}} = \min(f \text{ dei nodi ritagliati})$
4. Ripete finché trova il goal o nessun nodo è ritagliabile

Vantaggio: memoria $O(b \cdot d)$ invece di $O(b^d)$ come A*. **Svantaggio:** riesplora i nodi ad ogni iterazione.

```

1 idastar(Domain, Start, Path, Cost, ITERS) :-
2     heuristic(Domain, Start, H0),
3     idastar_loop(Domain, Start, H0, Path, Cost, 0, ITERS).
4
5 idastar_loop(Domain, Start, Bound, Path, Cost, ITERS0, ITERS) :-
6     idastar_search(Domain, Start, 0, Bound, [Start], ResultPath,
7         ResultCost, NextBound),
8     (ResultPath \= failure ->
9         Path = ResultPath, Cost = ResultCost, ITERS is ITERS0 + 1
10    ;
11        ITERS1 is ITERS0 + 1,
12        idastar_loop(Domain, Start, NextBound, Path, Cost, ITERS1, ITERS)
13    ).
14
15 % Caso base: goal trovato
16 idastar_search(Domain, State, G, _, RevPath, Path, G, _) :-
17     goal(Domain, State), reverse(RevPath, Path), !.
18
19 % Caso ricorsivo: cutoff o espansione
20 idastar_search(Domain, State, G, Bound, RevPath, Path, Cost, NextBound) :-

```

```

20 heuristic(Domain, State, H), F is G + H,
21 (F > Bound ->                                     % CUTOFF: f supera la soglia
22     Path = failure, Cost = 0, NextBound = F
23 ;
24     findall(s(S1, SC),
25         (move(Domain, State, S1, SC),
26             \+ member(S1, RevPath)),           % evita cicli (path checking)
27         Succs),
28     search_successors(Domain, Succs, G, Bound, RevPath, Path, Cost,
29         1.0e308, NextBound)
30 ).

```

6.4 Interfaccia Generica

```

1 % Collegamento domain -> predicati specifici
2 move(maze, S, S1, C) :- maze_move(S, S1, C).
3 move(puzzle, S, S1, C) :- puzzle_move(S, S1, C).
4
5 heuristic(maze, S, H) :- maze_heuristic(S, H).
6 heuristic(puzzle, S, H) :- puzzle_heuristic(S, H).
7
8 goal(maze, S) :- maze_goal(S).
9 goal(puzzle, S) :- puzzle_goal_check(S).

```

Aggiungere un nuovo dominio richiede solo definire tre predicati: `move(nuovo, ...)`, `heuristic(nuovo, ...)`, `goal(nuovo, ...)`.

6.5 Dominio 1: Labirinto (5×5)

```

1 % Stato: pos(Riga, Colonna). Muri: wall(R, C). Uscite: maze_exit(pos(R,C))
2 .
3 maze_move(pos(R, C), pos(R1, C), 1) :-          % SU
4     R1 is R - 1, maze_size(MaxR, _),
5     R1 >= 1, R1 =< MaxR, \+ wall(R1, C).
6 % ... analoghe per GIU', SINISTRA, DESTRA
7
8 maze_heuristic(pos(R, C), H) :-                  % Manhattan verso uscita piu'
9     vicina
10     findall(D, (maze_exit(pos(Re, Ce)), D is abs(R-Re)+abs(C-Ce)), Ds),
11     min_list(Ds, H).

```

Ammissibilità della Manhattan: nei movimenti ortogonali con costo 1, la Manhattan è il numero minimo di passi senza ostacoli. I muri possono solo aumentare il costo reale, quindi $h \leq h^*$ sempre.

6.6 Dominio 2: Puzzle dell'8

Stato: lista di 9 elementi [0,1,2,3,4,5,6,7,8], lo 0 è la casella vuota.

```

1 % Mapping indice -> (riga, colonna):
2 %   Riga = Indice div 3,   Colonna = Indice mod 3
3
4 puzzle_move(State, NewState, 1) :-
5     blank_pos(State, Blank),

```

```

6      (   Blank >= 3, Swap is Blank - 3    % SU
7      ;   Blank <= 5, Swap is Blank + 3    % GIU'
8      ;   Col is Blank mod 3, Col > 0, Swap is Blank - 1  % SINISTRA
9      ;   Col is Blank mod 3, Col < 2, Swap is Blank + 1  % DESTRA
10     ),
11     swap_elements(State, Blank, Swap, NewState).
12
13 % Euristic: somma delle distanze Manhattan di tutte le tessere
14 puzzle_heuristic(State, H) :-
15     puzzle_goal(Goal),
16     findall(D,
17         (nth0(Idx, State, Tile), Tile \= 0,
18          nth0(GoalIdx, Goal, Tile),
19          R1 is Idx // 3, C1 is Idx mod 3,
20          R2 is GoalIdx // 3, C2 is GoalIdx mod 3,
21          D is abs(R1-R2) + abs(C1-C2)),
22         Ds),
23     sumlist(Ds, H).

```

Puzzle dell'8: condizione di risolubilità

Non tutte le permutazioni di $[0..8]$ sono risolubili! Una permutazione è risolubile **se e solo se il numero di inversioni è pari**. Un'inversione è una coppia (A, B) con A prima di B , $A > B$, e nessuno dei due è 0. Esattamente metà delle 362880 permutazioni è risolubile.

6.7 Confronto A* vs IDA*

Proprietà	A*	IDA*
Memoria	$O(b^d)$ – esponenziale	$O(b \cdot d)$ – lineare
Nodi riesplorati	No	Sì (ogni iterazione)
Struttura dati	Open list ordinata + Closed list	Solo stack di ricorsione
Cicli	Gestiti dalla Closed list	Gestiti con path checking
Ottimale	Sì (con h ammissibile)	Sì (con h ammissibile)
Completo	Sì	Sì (con costi > 0)
Statistiche	Nodi espansi + max open	Numero iterazioni

Parte II

Answer Set Programming (Clingo)

7 Fondamenti di ASP

7.1 Paradigma

L'ASP è un paradigma **dichiarativo**: si descrivono i **vincoli** che la soluzione deve soddisfare. Il solver (Clingo) trova gli **answer set** (modelli stabili) che soddisfano tutti i vincoli.

Pipeline:

Grounding (variabili $\rightarrow$ costanti) $\rightarrow$ Solving (cerca modelli stabili) $\rightarrow$ Ottimizzazione

7.2 Costrutti Fondamentali

Costrutto	Sintassi	Significato
Fatto	<code>p(a).</code>	<code>p(a)</code> è sempre vero
Regola normale	<code>h :- b1, b2.</code>	derivazione condizionale
Negazione NAF	<code>not p</code>	vero se <code>p</code> non è nell'answer set
Choice rule	<code>{h} :- body.</code>	<code>h</code> può o non può essere incluso
Vincolo hard	<code>:- body.</code>	elimina modelli dove <code>body</code> è vero
Aggregato count	<code>#count{x : p(x)}</code>	conta gli <code>x</code> con <code>p(x)</code>
Aggregato sum	<code>#sum{w,k : p(k,w)}</code>	somma dei pesi
Modulo	<code>N \ M</code>	resto intero di <code>N/M</code>
Minimizzazione	<code>#minimize{w,k : p}.</code>	minimizza la somma
Show	<code>#show p/n.</code>	mostra solo <code>p</code> nell'output

Choice Rule: la distinzione chiave da Prolog

In Prolog, se il corpo è vero la testa *deve* essere vera. In ASP, `{h} :- body` significa che `h` *può* essere vero se `body` è vero, ma non è obbligatorio. Il solver esplora entrambe le possibilità e mantiene i modelli che soddisfano i vincoli.

Vincolo Hard: il filtro

`:- body` elimina dall'insieme di answer set tutti i modelli dove `body` è vero. Equivale a dire: “è **vietato** che `body` sia soddisfatto”.

8 Progetto Torneo Pokémon – torneo.lp**8.1 Sistema Svizzero**

Regole di abbinamento:

1. Giocatori con punteggio simile si affrontano (spareggio).
2. Nessun rematching (stessa coppia non si affronta due volte).
3. Se numero dispari: un giocatore riceve il BYE (vittoria auto). Nessuno lo prende due volte.

8.2 Normalizzazione: pid e played_already

```

1 % Simmetria degli incontri
2 played_already(P1, P2) :- played(P1, P2).
3 played_already(P1, P2) :- played(P2, P1).
4
5 % Indice canonico: quanti giocatori vengono prima di P lessicograficamente
6 pid(P, I) :- player(P),
7           I = #count{ Q : player(Q), Q != P, Q < P }.

```

Scopo di pid: assegnare un ordine totale ai giocatori per generare ogni coppia {P1, P2} in una sola forma canonica (solo quando $\text{pid}(P1) < \text{pid}(P2)$), evitando di avere sia $\text{match}(A,B)$ che $\text{match}(B,A)$.

8.3 Generazione

```

1 % match solo in forma canonica (pid(P1) < pid(P2))
2 { match(P1, P2) } :-
3     player(P1), player(P2),
4     pid(P1, I1), pid(P2, I2), I1 < I2,
5     not played_already(P1, P2).
6
7 % BYE solo per chi non lo ha mai avuto
8 { bye(P) } :- player(P), not had_bye(P).

```

8.4 Vincoli Hard

```

1 in_match(P) :- match(P, _).
2 in_match(P) :- match(_, P).
3
4 % Ogni giocatore: esattamente match OR bye
5 :- player(P), not in_match(P), not bye(P). % deve fare qualcosa
6 :- player(P), in_match(P), bye(P). % non entrambi
7
8 % Al massimo in un match (3 casi di conflitto)
9 :- match(P, Q1), match(P, Q2), Q1 != Q2. % P in due match come P1
10 :- match(P1, Q), match(P2, Q), P1 != P2. % Q in due match come P2
11 :- match(P, _), match(_, P). % P in entrambi i ruoli
12
13 % Un solo BYE per turno
14 :- 2 <= #count{ P : bye(P) }.
15
16 % Parita': BYE iff numero dispari
17 player_count(N) :- N = #count{ P : player(P) }.
18 :- player_count(N), 0 == N \ 2, bye(_). % pari -> no BYE
19 :- player_count(N), 1 == N \ 2, not bye(_). % dispari -> BYE obbligatorio

```

8.5 Ottimizzazione

```

1 score_diff(P1, P2, D) :-
2     match(P1, P2), score(P1, S1), score(P2, S2),
3     S1 >= S2, D = S1 - S2.
4 score_diff(P1, P2, D) :-
5     match(P1, P2), score(P1, S1), score(P2, S2),
6     S2 > S1, D = S2 - S1.

```

```

7
8 #minimize { D, P1, P2 : score_diff(P1, P2, D) }.

```

Minimizza $\sum |S_{P1} - S_{P2}|$ su tutti i match. Match tra giocatori con punteggio uguale contribuiscono 0 alla somma (preferiti).

8.6 Istanze

Turno	Giocatori	Parità	BYE	Ottimiz.
Turno 1	5 (dispari)	$5 \bmod 2 = 1$	Obbligatorio	Tutti a 0: qualsiasi match
Turno 2	8 (pari)	$8 \bmod 2 = 0$	Vietato	Abbina i "2" tra loro

Parte III

CLIPS – Expert System

9 Fondamenti di CLIPS

9.1 Cos'è CLIPS

CLIPS (C Language Integrated Production System) è un framework per **sistemi esperti** basato su regole di produzione.

Componente	Nome CLIPS	Funzione
Fatti	Working Memory (WM)	Ciò che il sistema “sa”
Regole	Production Rules	if condizioni then azioni
Motore	Inference Engine	Seleziona e spara le regole

9.2 Ciclo di Inferenza

1. **Match**: trova tutte le regole le cui condizioni (LHS) sono soddisfatte dai fatti in WM $\rightarrow$ *agenda*.
2. **Conflict resolution**: seleziona la regola con **salience** maggiore.
3. **Fire**: esegue le azioni (RHS) della regola (assert, retract, modify, focus, ...).
4. Ripete finché l'agenda è vuota.

9.3 Costrutti Base

```

1 ;;; Template (struttura dati)
2 (deftemplate MAIN::cell
3   (slot x (type INTEGER))
4   (slot visited (type SYMBOL) (default no))
5   (slot pit-cf (type FLOAT) (default 0.0))
6 )
7
8 ;;; Regola
9 (defrule MAIN::nome-regola
10  (declare (salience 10)) ; priorit  (default 0)
11  (percept (x ?x) (y ?y)) ; condizioni (LHS)

```

```

12   ?c <- (cell (x ?x))           ; bind del fatto a ?c
13 =>
14   (modify ?c (visited yes))      ; azione: modifica ?c
15   (assert (nuova-cosa))          ; aggiungi un fatto
16 )
17
18 ;;; Modulo
19 (defmodule PERCEPT (import MAIN ?ALL) (export ?ALL))

```

Saliency

La saliency determina l'ordine di esecuzione delle regole. Range: -10000 a $+10000$, default 0. Regole con saliency più alta sparano prima.

Moduli e Focus

Un modulo è un namespace per regole. (focus MODULO) mette il modulo in cima allo stack: solo le sue regole possono sparare. Quando il modulo esaurisce le regole attive, si torna al modulo precedente nello stack.

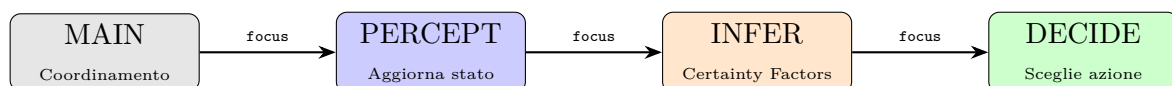
10 Progetto Wumpus – agent.clp + wumpus.py

10.1 Architettura Generale

Python simula il mondo (griglia, Wumpus, pozzi, oro) e fa da bridge:

1. Calcola le percezioni dalla posizione corrente.
2. Inietta un fatto (percept ...) in CLIPS.
3. Chiama `env.run()` per far sparare le regole.
4. Legge il fatto (action ...) e applica l'azione al mondo.

10.2 Moduli dell'Agente CF



10.3 Template Principali

```

1 (deftemplate MAIN::cell
2   (slot x) (slot y)
3   (slot visited (type SYMBOL) (default no))
4   (slot safe (type SYMBOL) (default unknown)) ; yes/no/unknown
5   (slot pit-cf (type FLOAT) (default 0.0)) ; in [-1, +1]
6   (slot wumpus-cf (type FLOAT) (default 0.0))
7 )
8 (deftemplate MAIN::percept ; iniettato da Python ogni ciclo
9   (slot x) (slot y) (slot steps) (slot energy)
10  (slot breeze (type SYMBOL)) ; yes/no
11  (slot stench (type SYMBOL)) ; yes/no
12  (slot glitter (type SYMBOL)) ; yes/no
13 )
14 (deftemplate MAIN::pending-action
15   (slot move (type SYMBOL))

```

```

16 (slot priority (type INTEGER) (default 0))
17 )
18 (deftemplate MAIN::cf-done ; impedisce loop da modify
19 (slot x) (slot y) (slot kind) ; kind = pit o wumpus
20 )

```

10.4 Certainty Factors (CF)

Teoria dei Certainty Factors

Un $CF \in [-1, +1]$ quantifica la certezza di un'ipotesi:

- -1 : certezza assoluta che sia **falsa**
- 0 : completa ignoranza
- $+1$: certezza assoluta che sia **vera**

Evidenza positiva (brezza/odore rilevato, $\alpha = 0.5$):

$$CF_{\text{new}} = CF + (1 - CF) \times 0.5$$

Evidenza negativa (nessuna brezza/odore, $\alpha = 0.7$):

$$CF_{\text{new}} = CF + (-1 - CF) \times 0.7$$

Le formule hanno rendimento decrescente: il CF non supera mai ± 1 . Partendo da 0, dopo 1 evidenza positiva: 0.5; dopo 2: 0.75; dopo 3: 0.875. Dopo 1 evidenza negativa: -0.7 ; dopo 2: -0.91 .

10.4.1 Esempio di regola CF – INFER

```

1 (defrule INFER::breeze-north
2   (declare (salience 10))
3   (percept (x ?x) (y ?y) (breeze yes))
4   ?c <- (cell (x ?cx) (y ?cy) (visited no) (pit-cf ?cf))
5   (test (= ?cx ?x))
6   (test (= ?cy (+ ?y 1))) ; cella a nord
7   (not (cf-done (x ?cx) (y ?cy) (kind pit))) ; non già' aggiornata
8 =>
9   (assert (cf-done (x ?cx) (y ?cy) (kind pit))) ; blocca ri-
10      attivazione
11   (modify ?c (pit-cf (+ ?cf (* (- 1.0 ?cf) 0.5)))) ; CF_new = CF +
12      (1-CF)*0.5
13 )

```

Il problema del loop da modify

(modify ?f ...) in CLIPS fa retract + re-assert del fatto, rendendo il fatto “nuovo” e riattivando tutte le regole che lo matchano. **Senza cf-done**: la regola sparerebbe infinite volte sulla stessa cella. **Con cf-done**: la condizione (not (cf-done ...)) blocca la ri-attivazione nello stesso ciclo.

10.4.2 Classificazione delle Celle

```

1 (defrule INFER::mark-safe      ; pit-cf < -0.5 E wumpus-cf < -0.5 -> sicura
2   ?c <- (cell (visited no) (safe unknown) (pit-cf ?p) (wumpus-cf ?w))
3   (test (< ?p -0.5)) (test (< ?w -0.5))
4 => (modify ?c (safe yes)) )
5
6 (defrule INFER::mark-dangerous-pit ; pit-cf > 0.6 -> pericolosa
7   ?c <- (cell (visited no) (safe unknown) (pit-cf ?p))
8   (test (> ?p 0.6))
9 => (modify ?c (safe no)) )

```

10.5 Gerarchia delle Azioni – DECIDE

Priorità	Azione	Quando
100	grab	glitter presente
90	quit	energia ≤ 30
80	up/down/left/right	cella safe=yes non visitata adiacente
60	pit-north/...	CF pit > 0.3 e CF wumpus < 0.3 (indovina)
10	up/... (esplora)	CF pit < 0.5 e CF wumpus < 0.5
5	up/... (coraggioso)	cella non esplicitamente pericolosa
0	quit (fallback)	nessuna altra opzione

Meccanismo di selezione: le regole asseriscono pending-action con una priorità. Poi cleanup-lower rimuove quelle subottimali e emit-action promuove la vincente ad action.

```

1 (defrule DECIDE::cleanup-lower
2   (declare (salience 0))
3   ?pa <- (pending-action (priority ?p1))
4   (pending-action (priority ?p2))
5   (test (> ?p2 ?p1))
6 => (retract ?pa) )
7
8 (defrule DECIDE::emit-action
9   (declare (salience -1))
10  (not (action))
11  ?pa <- (pending-action (move ?m) (priority ?p))
12  (not (pending-action (priority ?p2&:(> ?p2 ?p))))
13 => (assert (action (move ?m))) (retract ?pa) )

```

10.6 Agente Greedy vs Agente CF

Aspetto	agent.clp (CF)	agent_greedy.clp
Moduli	4 (MAIN/PERCEPT/INFER/DECIDE)	1 (MAIN)
Cella pericolosa	CF $\in [-1, +1]$, graduabile	dangerous = yes/no (binario)
Aggiornamento	Incrementale, reversibile	Irreversibile (1 brezza = pericolo)
Pit-guess	Sì (se CF moderato)	No
Blocco	Raramente (ha fallback multipli)	Spesso (tutte le celle dangerous)

Parte IV

Schema Riassuntivo per l'Esame

11 Confronto tra i Paradigmi

Aspetto	Prolog	ASP (Clingo)	CLIPS
Paradigma	Logico / dichiarativo	Logico / constraint	Regole di produzione
Negazione	NAF ($\backslash +$)	NAF (not)	Patterns negativi (not)
Scelta	Backtracking	Choice rules {h}	Agenda (salience)
Vincoli	Clausole Horn	Integrity constraints	Condizioni LHS
Ottimizzazione	No (built-in)	#minimize/maximize	No (built-in)
Uso principale	Ricerca, logica, NLP	Combinatoria, pianificazione	Sistemi esperti

12 Domande di Esame Tipiche

12.1 Su Prolog

1. **Cos'è il backtracking e quando si attiva?** Quando un goal fallisce o si vuole la prossima soluzione, Prolog torna all'ultimo punto di scelta e prova l'alternativa successiva.
2. **Differenza tra = e is?** $X = 3+2$ unifica X con il termine $+(3,2)$ (non valuta). $X \text{ is } 3+2$ valuta l'espressione e unifica X con 5.
3. **Cosa fa il cut (!)?** Elimina i punti di backtracking: impedisce di tornare indietro sulla clausola corrente e sulle sue alternative.
4. **Differenza tra cut verde e rosso?** Verde: solo efficienza, non cambia il significato. Rosso: cambia il significato, elimina soluzioni che senza cut sarebbero trovate.

5. **Perché findall non fallisce mai?** Perché se il goal non ha soluzioni restituisce [].
6. **A* è ottimale? Quando?** Sì, se l'euristica è **ammissibile**: $h(n) \leq h^*(n)$ per ogni n .
7. **Vantaggio principale di IDA* su A*?** Memoria lineare $O(b \cdot d)$ invece di esponenziale $O(b^d)$.
8. **Perché la distanza Manhattan è ammissibile?** Conta i passi minimi senza ostacoli. I muri possono solo aumentare il costo reale, quindi non sovrastima mai.

12.2 Su ASP/Clingo

1. **Differenza tra regola normale e choice rule?** Normale ($h :- \text{body}$): se body è vero, h **deve** essere vero. Choice ($\{h\} :- \text{body}$): se body è vero, h **può** essere vero.
2. **Cosa fa un integrity constraint?** $:- \text{body}$ elimina tutti i modelli dove body è vero.
3. **Perché si usa pid nel torneo?** Per generare ogni coppia di giocatori in una sola forma canonica (evitare sia $\text{match}(A,B)$ sia $\text{match}(B,A)$).
4. **Cosa minimizza #minimize?** La somma dei pesi degli elementi dell'insieme che soddisfano la condizione. Nel torneo: somma delle differenze di punteggio tra gli abbinati.
5. **Come si gestisce la parità nel torneo?** Con l'operatore modulo $N \setminus 2$: se resto 0 (pari) si vieta il BYE; se resto 1 (dispari) si impone il BYE.

12.3 Su CLIPS

1. **Cos'è la salience?** Un valore intero che determina la priorità di una regola. Valori più alti sparano prima. Default: 0.
2. **Perché si usa cf-done?** Per evitare loop infiniti causati da **modify**: ogni **modify** ri-asserta il fatto rendendolo nuovo, riattivando le regole. Il fatto **cf-done** blocca la ri-attivazione nello stesso ciclo.
3. **Cosa sono i Certainty Factors?** Valori in $[-1, +1]$ che quantificano la certezza di un'ipotesi. Si aggiornano con formule che combinano evidenza positiva e negativa.
4. **Perché l'agente CF è superiore al greedy?** Perché gestisce l'incertezza in modo graduato: può rivalutare una cella parzialmente pericolosa se nuove evidenze negative la abbassano. Il greedy è irreversibile: una singola brezza marca la cella come pericolosa per sempre.
5. **Perché l'action si rimuove prima di iniettare il percept?** Perché la regola **handle-percept** si attiva solo quando (**not (action)**). Se l'action del ciclo precedente rimane in WM, il nuovo ciclo non parte.

13 Formule da Ricordare

Formula	Contesto	Significato
$f(n) = g(n) + h(n)$	A* / IDA*	costo totale stimato attraverso n
$h(n) \leq h^*(n)$	Ammissibilità	condizione per ottimalità
$h_{\text{Manhattan}} = r_1 - r_2 + c_1 - c_2 $	Labirinto / Puzzle	stima del percorso
$CF_{\text{new}} = CF + (1 - CF) \times 0.5$	CLIPS / CF positivo	aggiornamento con brezza
$CF_{\text{new}} = CF + (-1 - CF) \times 0.7$	CLIPS / CF negativo	aggiornamento senza brezza
$N \bmod 2 = 0 \Rightarrow \text{no BYE}$	ASP Torneo	vincolo di parità